Mahatma Education Society's

Pillai College of Arts, Commerce & Science

(Autonomous)

Affiliated to University of Mumbai

NAAC Accredited 'A' grade (3 cycles)

Best College Award by University of Mumbai

ISO 9001:2015 Certified

CERTIFICATE

This is to certify that Mr. ***Rohit Malviya*** *of* ***M.Sc. D.A. Part-II Semester IV*** *has completed the practical work in the Subject of* ***Deep Learning*** *during the academic year 2024-25 under the guidance of Prof.* ***Omkar Sherkhane*** *being the partial requirement for the fulfillment of the curriculum of* ***Degree of Master of Science in Data Analytics, University of Mumbai.***

Place:

Date:

Name & Signature of faculty *Name & Signature of external*

INDEX

Sr. no.	Details	Date	Signature
1.	Artificial Neural Network	05/11/25	
2.	Convolutional Neural Network	06/11/25	
3.	Recurrent Neural Network	11/11/25	
4.	LSTM	13/11/25	
5.	Linear Regression	13/11/25	
6.	Object Detection	05/01/26	
7.	Face Detection and Recognition	09/01/25	
8.	Tensor flow operations	29/11/25	
9.	Simple Example on Reinforcement Learning	23/01/25	
10.	To study Reinforcement Learning Concept on Cart-Pole with DQN	23/01/25	

PRACTICAL 1

AIM: Artificial Neural Network

Code:-

```
[2]
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_classification
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# 1. Generate a synthetic dataset
X, y = make_classification(n_samples=1000, n_features=20, n_informative=15, n_redundant=5, random_state=42)

# 2. Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Scale the features (Important for ANNs)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print(f"Training set shape: {X_train.shape}")

Training set shape: (800, 20)
```

```
[10]
✓ 0s
# 4. Build the ANN model
model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])

# 5. Compile the model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
dense_9 (Dense)	(None, 32)	672
dense_10 (Dense)	(None, 16)	528
dense_11 (Dense)	(None, 1)	17

Total params: 1,217 (4.75 KB)
Trainable params: 1,217 (4.75 KB)
Non-trainable params: 0 (0.00 B)

[11]
10s

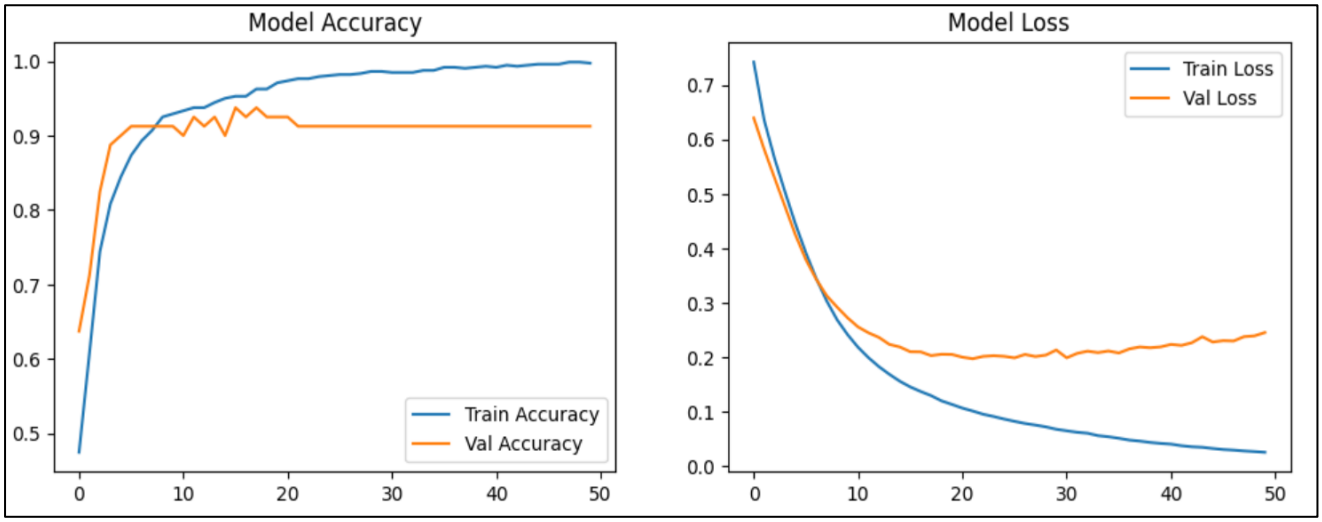
```python
# 6. Train the model
history = model.fit(X_train, y_train,
                    epochs=50,
                    batch_size=32,
                    validation_split=0.1,
                    verbose=1)

# 7. Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)
print(f'\nTest Accuracy: {accuracy:.4f}')
```

```
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9736 - loss: 0.0972 - val_accuracy: 0.9500 - val_loss: 0.1590
Epoch 24/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9778 - loss: 0.0936 - val_accuracy: 0.9375 - val_loss: 0.1588
Epoch 25/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9792 - loss: 0.0866 - val_accuracy: 0.9375 - val_loss: 0.1581
Epoch 26/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9750 - loss: 0.0828 - val_accuracy: 0.9375 - val_loss: 0.1572
Epoch 27/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9806 - loss: 0.0783 - val_accuracy: 0.9375 - val_loss: 0.1554
Epoch 28/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9764 - loss: 0.0765 - val_accuracy: 0.9500 - val_loss: 0.1590
Epoch 29/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9819 - loss: 0.0717 - val_accuracy: 0.9375 - val_loss: 0.1528
Epoch 30/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9833 - loss: 0.0674 - val_accuracy: 0.9500 - val_loss: 0.1544
```

```
Epoch 49/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 1.0000 - loss: 0.0257 - val_accuracy: 0.9500 - val_loss: 0.1641
Epoch 50/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 1.0000 - loss: 0.0246 - val_accuracy: 0.9500 - val_loss: 0.1625
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9250 - loss: 0.1935

Test Accuracy: 0.9250
```

[5]
0s

```python
# 8. Visualize training history
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Model Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Model Loss')
plt.legend()

plt.show()
```



PRACTICAL 2

AIM: Convolutional Neural Network

Code:-

- **Using in-built dataset**

[1] 12s

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
```

[2] 0s

```
(x_train,y_train),(x_test,y_test) = datasets.mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

[3] 0s

```
#Reshaping & normalizing into matrix of -1,1

x_train=x_train.reshape((x_train.shape[0],28,28,1)).astype('float32')/255
x_test=x_test.reshape((x_test.shape[0],28,28,1)).astype('float32')/255
```

[4] 0s

```
#Building the CNN model

model = models.Sequential([
    layers.Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(64,activation='relu'),
    layers.Dense(10,activation='softmax')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: Use
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

[5] 0s

```
#Compiling the model

model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
```

[6]
6m

```
model.fit(x_train,y_train,epochs=5,validation_data=(x_test,y_test))
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 59s 30ms/step - accuracy: 0.9563 - loss: 0.1405 - val_accuracy: 0.9850 - val_loss: 0.0490
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 81s 30ms/step - accuracy: 0.9855 - loss: 0.0467 - val_accuracy: 0.9883 - val_loss: 0.0364
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 82s 30ms/step - accuracy: 0.9901 - loss: 0.0318 - val_accuracy: 0.9894 - val_loss: 0.0339
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 55s 29ms/step - accuracy: 0.9924 - loss: 0.0237 - val_accuracy: 0.9894 - val_loss: 0.0318
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 82s 29ms/step - accuracy: 0.9939 - loss: 0.0187 - val_accuracy: 0.9915 - val_loss: 0.0276
<keras.src.callbacks.history.History at 0x7f1575f2d9a0>
```

[7]
2s

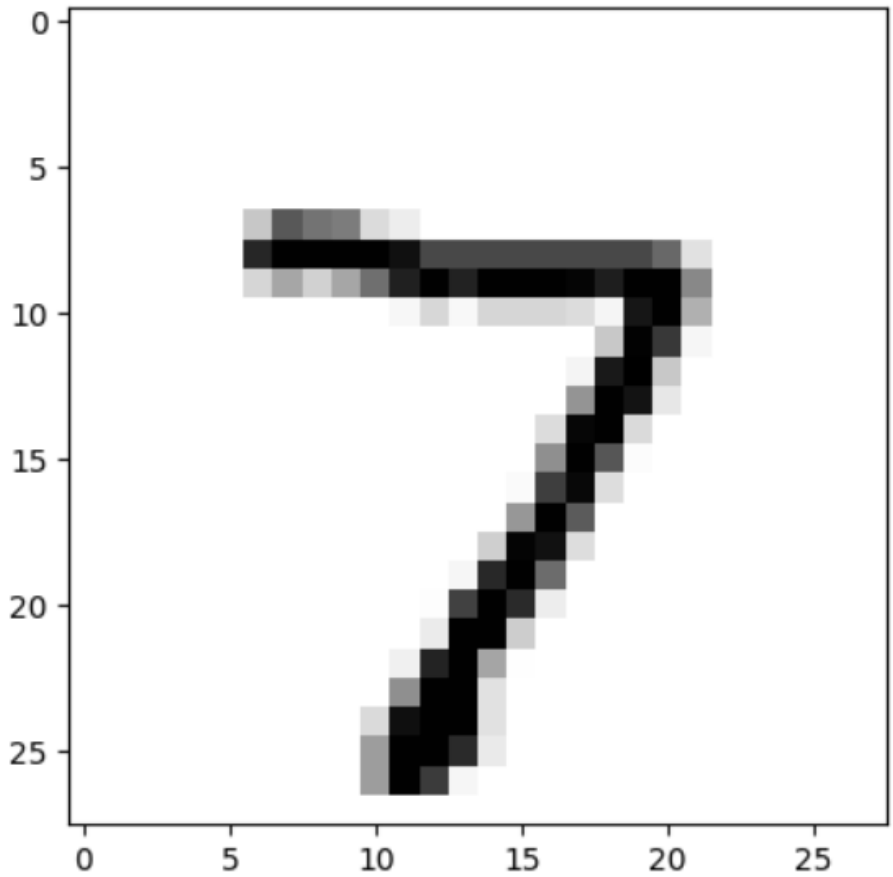
```
#Evaluating the model

test_loss, test_acc=model.evaluate(x_test,y_test, verbose=2)
print(f"test_acc:{test_acc:.2f}")
```

```
313/313 - 3s - 8ms/step - accuracy: 0.9915 - loss: 0.0276
test_acc:0.99
```

[8]
0s

```
import numpy as np
import matplotlib.pyplot as plt
image_index=0
test_image=x_test[image_index]
plt.imshow(test_image.reshape(28,28),cmap='Greys')
```

```
<matplotlib.image.AxesImage at 0x7f15b483ca10>
```



- **Using downloaded dataset**

Cat & Dog Classification Using CNN

[9] ✓ 0s

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
```

[10] ✓ 0s

```
train_dir = "/content/drive/MyDrive/training_set"
test_dir = "/content/drive/MyDrive/test_set"
```

[11] ✓ 14s

```
# Step 2: Image Preprocessing
# ----------------------------
train_datagen = ImageDataGenerator(
    rescale=1.0/255,
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True
)

test_datagen = ImageDataGenerator(rescale=1.0/255)

train_set = train_datagen.flow_from_directory(
    train_dir,
    target_size=(128, 128),
    batch_size=32,
    class_mode='binary'
)

test_set = test_datagen.flow_from_directory(
    test_dir,
    target_size=(128, 128),
    batch_size=32,
    class_mode='binary'
)
```

```
Found 8005 images belonging to 1 classes.
Found 2023 images belonging to 1 classes.
```

[12]
✓ 0s

```
# Step 3: Build the CNN model
# ----------------------------
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(128, 128, 3)),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(128, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')  # Binary classification
])
```

[13]
✓ 0s

```
# Step 4: Compile the model
# ----------------------------
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

[14]
✓ 38m

```
# Step 5: Train the model
# ----------------------------
history = model.fit(
    train_set,
    epochs=3,
    validation_data=test_set
)
```

```
Epoch 1/3
251/251 ━━━━━━━━━━━━━━━━━━━━ 1623s 6s/step - accuracy: 0.9980 - loss: 0.0027 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
Epoch 2/3
251/251 ━━━━━━━━━━━━━━━━━━━━ 331s 1s/step - accuracy: 1.0000 - loss: 3.1070e-29 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
Epoch 3/3
251/251 ━━━━━━━━━━━━━━━━━━━━ 329s 1s/step - accuracy: 1.0000 - loss: 8.9161e-33 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
```

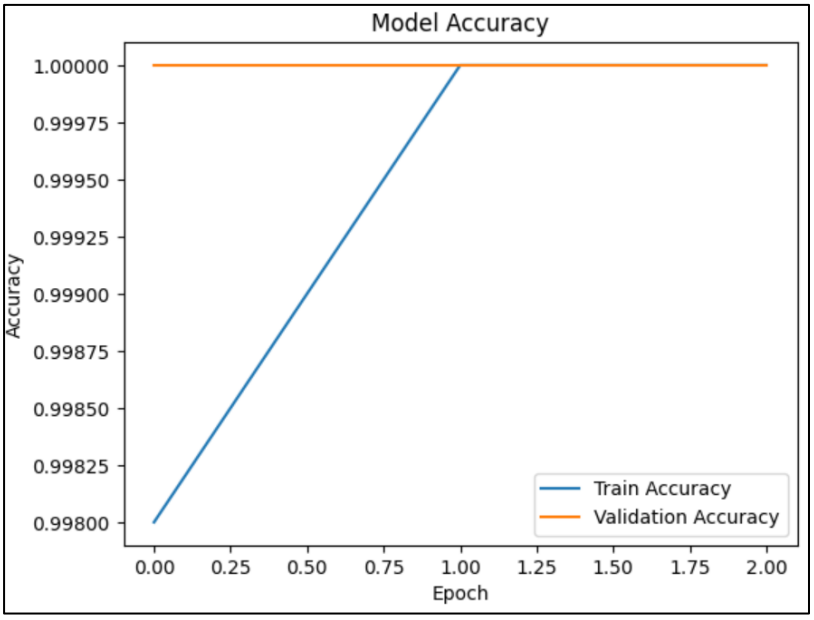
[15]
✓ 24s

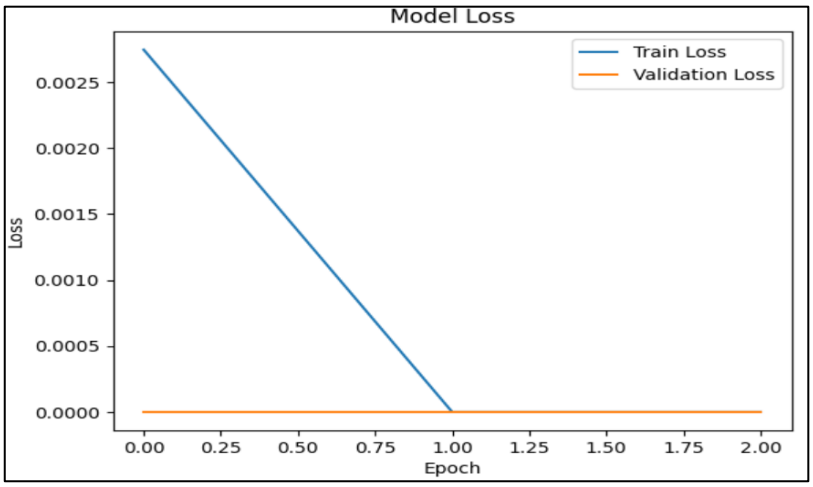
```
# ----------------------------
# Step 6: Evaluate the model
# ----------------------------
loss, accuracy = model.evaluate(test_set)
print(f"Test Accuracy: {accuracy*100:.2f}%")
```

```
64/64 ━━━━━━━━━━━━━━━━━━━━ 24s 376ms/step - accuracy: 1.0000 - loss: 0.0000e+00
Test Accuracy: 100.00%
```

[16]
✓ 0s

```
# ----------------------------
# Step 7: Plot training results
# ----------------------------
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```





[18]
✓ 0s

```python
# Step 8: Predict a Single Image (Cat/Dog)

import numpy as np
from tensorflow.keras.preprocessing import image

# ----------------------------
# Load and preprocess the image
# ----------------------------
img_path = "/content/drive/MyDrive/test_set/test_set/cats/cat.4001.jpg"  # change path

img = image.load_img(img_path, target_size=(128, 128))
img_array = image.img_to_array(img)
img_array = img_array / 255.0   # rescale like training data
img_array = np.expand_dims(img_array, axis=0)  # shape (1, 128, 128, 3)

# ----------------------------
# Make prediction
# ----------------------------
prediction = model.predict(img_array)

# ----------------------------
# Interpret result
# ----------------------------
if prediction[0][0] > 0.5:
    result = "Dog"
else:
    result = "Cat"

print("Prediction:", result)

# ----------------------------
# Display the image
# ----------------------------
plt.imshow(img)
plt.title(f"Predicted: {result}")
plt.axis('off')
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 54ms/step
Prediction: Cat
```

Predicted: Cat

PRACTICAL 3

AIM: Recurrent Neural Network

Code:-

[1] 13s
```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense, Embedding
```

[2] 0s
```
text="hello world. this is rnn text generation."
```

[3] 0s
```
chars=sorted(set(text))
char2idx={c:i for i,c in enumerate(chars)}
idx2char={i:c for c,i in char2idx.items()}
```

[4] 0s
```
seq=[char2idx[c] for c in text]
```

[5] 0s
```
seq_length=10
X=[]
y=[]
for i in range(len(seq)-seq_length):
  X.append(seq[i:i+seq_length])          #I/P : 10 chars
  y.append(seq[i+seq_length])            #O/P : next char
```

[6] 0s
```
X=np.array(X)
y=tf.keras.utils.to_categorical(y,num_classes=len(chars))
```

[7] 0s
```
#Building the RNN model

model=Sequential([
    Embedding(input_dim=len(chars),output_dim=8,input_length=seq_length),
    SimpleRNN(64),
    Dense(len(chars),activation='softmax')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py
  warnings.warn(
```

[8] 0s
```
model.compile(loss='categorical_crossentropy',optimizer='adam')
```

[22] 5s
```
model.fit(X,y,epochs=100,verbose=0)
```

```
<keras.src.callbacks.history.History at 0x7cf9edf83230>
```

[23]
✓ 0s

```
#Function to generate text from a seed
def generate_text(seed,length=100):
  generated=seed
  for _ in range(length):
    input_seq=[char2idx.get(c,0) for c in generated[-seq_length:]]
    input_seq=np.pad(input_seq,(seq_length-len(input_seq),0))
    input_seq=np.expand_dims(input_seq,0)
    pred=model.predict(input_seq,verbose=0)
    next_char=idx2char[np.argmax(pred)]
    generated+=next_char
  return generated
```

[24]
✓ 8s

```
print(generate_text("hello wor"))
```

```
hello wornd. this en rnn t ntegeneignnnd.tn  .een r egnn.tegrnd.nernd.en rnten ntegenatennnd.tiratnen rati.nn
```

PRACTICAL 4

AIM: LSTM

Code:-

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
# Step 1: Define a sample paragraph (our tiny dataset)

text = "LSTMs are powerful models for sequence data. They can learn dependencies in text."
print("TEXT:", text)
```

```
TEXT: LSTMs are powerful models for sequence data. They can learn dependencies in text.
```

```
# Step 2: Tokenize words (convert words into integer IDs)

tokenizer = Tokenizer()
tokenizer.fit_on_texts([text])
word_index = tokenizer.word_index       # Dictionary mapping words -> integer IDs
print("WORD_INDEX:", word_index)        # Show the vocabulary mapping
sequences = tokenizer.texts_to_sequences([text])[0]  # Convert text into a list of integer IDs
print("SEQUENCES:", sequences)
```

```
WORD_INDEX: {'lstms': 1, 'are': 2, 'powerful': 3, 'models': 4, 'for': 5, 'sequence': 6, 'data': 7, 'they': 8, 'can': 9, 'learn': 10, 'depende
SEQUENCES: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]
```

```
# Step 3: Create input-output pairs (sliding window of 3 words -> next word)

X, y = [], []                           # Initialize input (X) and target (y) lists
window_size = 3
print("WINDOW_SIZE:", window_size)

for i in range(len(sequences) - window_size):
    X.append(sequences[i:i+window_size]) # Input: 3 consecutive words
    y.append(sequences[i+window_size])   # Target: the next word

print("X (INPUT WINDOWS):", X)
print("y (TARGETS):", y)

X = np.array(X)
y = np.array(y)
print("SHAPES:", X.shape, y.shape)
```

```
WINDOW_SIZE: 3
X (INPUT WINDOWS): [[1, 2, 3], [2, 3, 4], [3, 4, 5], [4, 5, 6], [5, 6, 7], [6, 7, 8], [7, 8, 9], [8, 9, 10], [9, 10, 11], [10, 11, 12]]
y (TARGETS): [4, 5, 6, 7, 8, 9, 10, 11, 12, 13]
SHAPES: (10, 3) (10,)
```

```python
# Step 4: Pad sequences (make sure all inputs are same length)

X = pad_sequences(X, maxlen=window_size) # Pad inputs to length 3 (already length 3 here)
print("PADDED X:\n", X)
```

```
PADDED X:
 [[ 1  2  3]
 [ 2  3  4]
 [ 3  4  5]
 [ 4  5  6]
 [ 5  6  7]
 [ 6  7  8]
 [ 7  8  9]
 [ 8  9 10]
 [ 9 10 11]
 [10 11 12]]
```

```python
# Step 5: Build the LSTM model

vocab_size = len(word_index) + 1         # Vocabulary size (+1 for padding index)
print("VOCAB_SIZE:", vocab_size)

model = keras.Sequential([               # Define a sequential model
    layers.Embedding(input_dim=vocab_size, output_dim=10, input_length=window_size), # Embedding maps word IDs to 10-d vectors
    layers.LSTM(32),                     # LSTM layer with 32 hidden units to learn sequence patterns
    layers.Dense(vocab_size, activation="softmax")  # Dense output layer with softmax to predict next word
])
print("MODEL SUMMARY:")
model.summary()
```

```
VOCAB_SIZE: 14
MODEL SUMMARY:
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

```python
# Step 6: Compile the model

model.compile(optimizer="adam", loss="sparse_categorical_crossentropy")  # Configure optimizer & loss
print("COMPILE STATUS: OK")
```

```
COMPILE STATUS: OK
```

```python
# Step 7: Train the model

history = model.fit(X, y, epochs=200, verbose=1)  # Train for 200 epochs with progress bar
print("TRAINING FINISHED")
```

```
Epoch 1/200
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step - loss: 2.6376
Epoch 2/200
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - loss: 2.6360
Epoch 3/200
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step - loss: 2.6344
Epoch 4/200
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step - loss: 2.6327
Epoch 5/200
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step - loss: 2.6311
Epoch 6/200
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step - loss: 2.6294
Epoch 7/200
```

```python
# Step 8: Predict the next word after ["LSTMs", "are", "powerful"]

test_seq = tokenizer.texts_to_sequences([["LSTMs", "are", "powerful"]])[0]  # Convert test words to IDs
print("TEST_SEQ IDS:", test_seq)

test_seq = pad_sequences([test_seq], maxlen=window_size)  # Pad test sequence to length 3
print("TEST_SEQ PADDED:\n", test_seq)

pred = model.predict(test_seq, verbose=0) # Predict probabilities for next word
print("PRED SHAPE:", pred.shape)

probabilities = pred[0]                  # Extract the 1D probability vector
print("TOP-5 (ID, PROB):", sorted(list(enumerate(probabilities)), key=lambda x: x[1], reverse=True)[:5])

predicted_id = int(np.argmax(pred))      # Index of maximum probability
predicted_word = tokenizer.index_word[predicted_id]  # Map ID back to word
print("PREDICTED NEXT WORD:", predicted_word)
```

```
TEST_SEQ IDS: [1, 2, 3]
TEST_SEQ PADDED:
 [[1 2 3]]
PRED SHAPE: (1, 14)
TOP-5 (ID, PROB): [(4, np.float32(0.80190736)), (5, np.float32(0.16371426)), (12, np.float32(0.011195118)
PREDICTED NEXT WORD: models
```

PRACTICAL 5

AIM: Linear Regression

Code:-

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GRU
from sklearn.preprocessing import MinMaxScaler
```

```
# Step 2: Simulating Stock Price Data

np.random.seed(42)
data=np.cumsum(np.random.randn(1000))+100
print(data[:5])
data=data.reshape(-1,1)
```

```
[100.49671415 100.35844985 101.00613839 102.52916825 102.29501487]
```

```
# Step 3: Normalizing the data

scaler=MinMaxScaler()
scaled_data=scaler.fit_transform(data)
print(scaled_data[:5])
```

```
[[0.38783024]
 [0.38470586]
 [0.39934175]
 [0.43375783]
 [0.42846664]]
```

```
# Step 4: Creating sequences

def create_sequences(data, seq_length):
    X,y=[],[]
    for i in range(len(data)-seq_length):
        X.append(data[i:i+seq_length])      #30 days of i/p
        y.append(data[i+seq_length])        #31st day as o/p
    return np.array(X),np.array(y)
```

```
seq_length=30
X,y=create_sequences(scaled_data,seq_length)

print("X shape: ",X.shape)
print("y shape: ",y.shape)
```

```
X shape:  (970, 30, 1)
y shape:  (970, 1)
```

```
# Step 5 : Building the GRU model

model=Sequential([
    GRU(64,input_shape=(seq_length,1)),
    Dense(1)
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/
  super().__init__(**kwargs)
```

```
model.compile(optimizer='adam',loss='mse')
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 64)	12,864
dense (Dense)	(None, 1)	65

Total params: 12,929 (50.50 KB)
Trainable params: 12,929 (50.50 KB)
Non-trainable params: 0 (0.00 B)

```
# Step 6: Training the model

model.fit(X,y,epochs=50,batch_size=32)
```

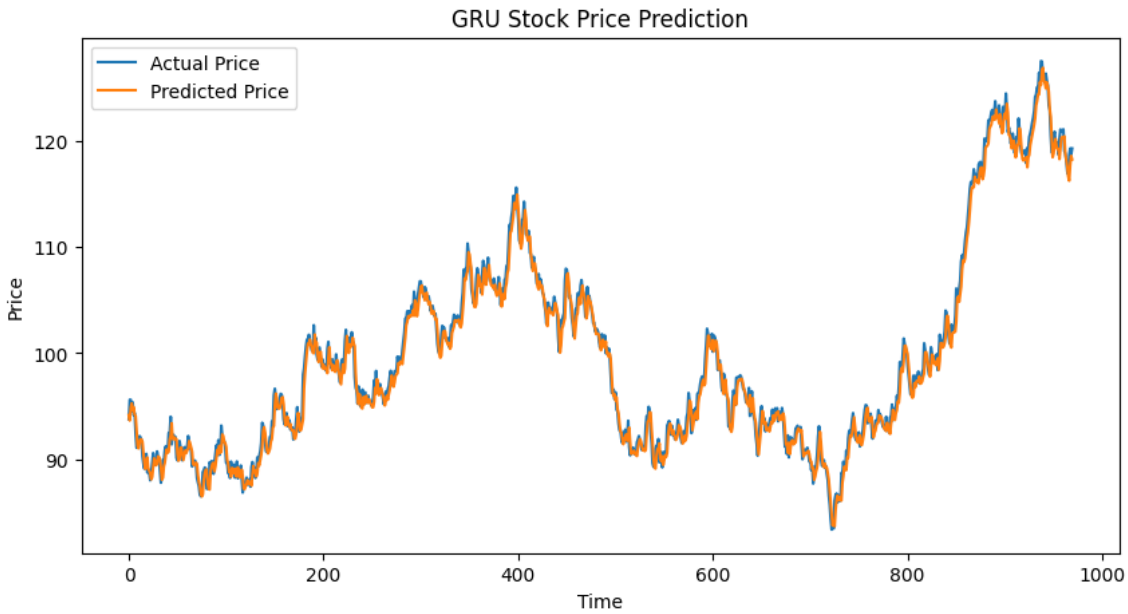
```
Epoch 1/50
31/31 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - loss: 0.0756
Epoch 2/50
31/31 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0059
Epoch 3/50
31/31 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0012
Epoch 4/50
31/31 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0010
Epoch 5/50
```

```
# Step 7: Making predictions

predicted=model.predict(X)

31/31 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
```

```
predicted_prices=scaler.inverse_transform(predicted)
actual_prices=scaler.inverse_transform(y)
```

```
print("Predicted: ", predicted_prices[:5].flatten())
print("Actual: ", actual_prices[:5].flatten())

Predicted:  [94.15615 93.63644 94.8398  95.26341 94.55628]
Actual:  [93.75388651 95.6061647  95.59266747 94.53495654 95.35750146]
```

```
#Step 8: Plotting the results

plt.figure(figsize=(10,5))
plt.plot(actual_prices,label='Actual Price')
plt.plot(predicted_prices,label='Predicted Price')
plt.legend()
plt.xlabel('Time')
plt.ylabel('Price')
plt.title('GRU Stock Price Prediction')
plt.show()
```




```
#Step 9: Predicting the o/p for 31st day

last_30_days=scaled_data[-30:].reshape(1,30,1)
next_day_scaled=model.predict(last_30_days)
next_day_price=scaler.inverse_transform(next_day_scaled)
print("Predicted price for 31st day: ",next_day_price[0][0])

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
Predicted price for 31st day:  118.6233
```

PRACTICAL 6

AIM: Object Detection

Code:-

```
pip install tensorflow

Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.9.23)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.7.0)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.0)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.1
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.2.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.76.0)
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.10.0)
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.15.1)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow) (0.45.1)
```

```
pip install tensorflow_hub

Requirement already satisfied: tensorflow_hub in /usr/local/lib/python3.12/dist-packages (0.16.1)
Requirement already satisfied: numpy>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow_hub) (2.0.2)
Requirement already satisfied: protobuf>=3.19.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow_hub) (5.29.5)
Requirement already satisfied: tf-keras>=2.14.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow_hub) (2.19.0)
Requirement already satisfied: tensorflow<2.20,>=2.19 in /usr/local/lib/python3.12/dist-packages (from tf-keras>=2.14.1->tensorflow_hub) (2.
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1->tens
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1->t
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->t
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1->t
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1->t
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1->tensorfl
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1->tensorf
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1->tensor
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1->te
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1->tens
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1->tenso
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow<2.20,>=2.19->tf-keras>=2.14.1->tenso
```

```
pip install opencv-python
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.12.0.88)
Requirement already satisfied: numpy<2.3.0,>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)
```

```
import tensorflow_hub as hub
import cv2
import numpy
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
```

```
detector = hub.load('https://tfhub.dev/tensorflow/efficientdet/lite2/detection/1')
```

```
width=1028
height=1028

img = cv2.imread('/content/sample_data/images.jpg')
inp = cv2.resize(img,(width,height))
rgb = cv2.cvtColor(inp,cv2.COLOR_BGR2RGB)
rgb_tensor=tf.convert_to_tensor(rgb,dtype=tf.uint8)
rgb_tensor=tf.expand_dims(rgb_tensor,0)
```

```
plt.figure(figsize=(10,10))
plt.imshow(rgb)
```

```
<matplotlib.image.AxesImage at 0x78e96e4c1370>
```

```
boxes,scores,classes,num_detections = detector(rgb_tensor)
```

```
labels = pd.read_csv('/content/sample_data/amazon_categories.csv')
labels = labels['category_name']
```

```
labels
```

	category_name
0	Beading & Jewelry Making
1	Fabric Decorating
2	Knitting & Crochet Supplies
3	Printmaking Supplies
4	Scrapbooking & Stamping Supplies
...	...
243	Xbox Series X & S Consoles, Games & Accessories
244	PC Games & Accessories
245	Baby Girls' Clothing & Shoes
246	Boys' School Uniforms
247	Toys & Games

248 rows × 1 columns

dtype: object

```
pred_labels = classes.numpy().astype('int')[0]
pred_labels = [labels[i] for i in pred_labels]
```

```
pred_boxes = boxes.numpy() [0].astype('int')
pre_scores = scores.numpy()[0]
```

```
for score, (ymin,xmin,ymax,xmax), label in zip(pre_scores,pred_boxes,pred_labels):
    if score < 0.5:
        continue

    score_txt = f'{100*round(score)}%'
    img_boxes = cv2.rectangle(rgb,(xmin,ymax),(xmax,ymin),(0,255,0),2)
    font = cv2.FONT_HERSHEY_SIMPLEX
    cv2.putText(img_boxes,label,(xmin,ymax-10),font,1.5,(255,0,0),2,cv2.LINE_AA)
    cv2.putText(img_boxes,score_txt,(xmax,ymax-10),font,1.5,(255,0,0),2,cv2.LINE_AA)
```

```
plt.figure(figsize=(10,10))
plt.imshow(img_boxes)

plt.savefig('image_pred.jpg',transparent=True)
```

primeday
0
200
400
600
800
1000
0
200
400
600
800
1000

PRACTICAL 7

AIM: Face Detection and Recognition

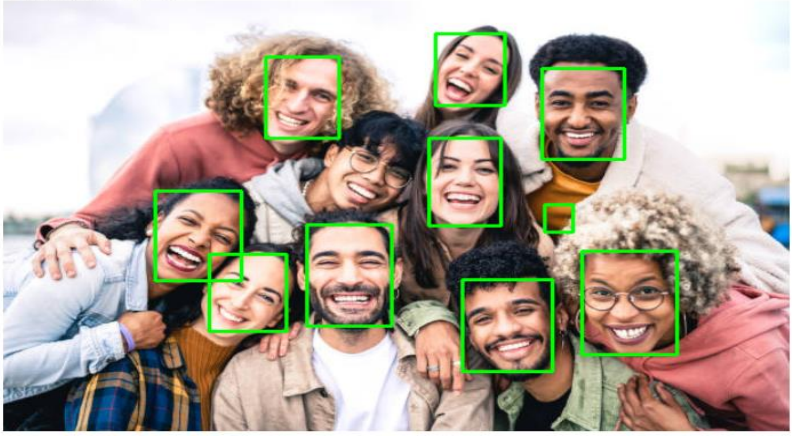
Code:-

```
pip install mtcnn opencv-python
```

```
Requirement already satisfied: mtcnn in /usr/local/lib/python3.12/dist-packages (1.0.0)
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.12.0.88)
Requirement already satisfied: joblib>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from mtcnn) (1.5.3)
Requirement already satisfied: lz4>=4.3.3 in /usr/local/lib/python3.12/dist-packages (from mtcnn) (4.4.5)
Requirement already satisfied: numpy<2.3.0,>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)
```

```
from google.colab.patches import cv2_imshow
import cv2
from mtcnn import MTCNN
detector = MTCNN()

image = cv2.imread('/content/images.jpg')
if image is None:
    raise FileNotFoundError("Image not found!")

rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
faces = detector.detect_faces(rgb_image)

print("Faces detected:", len(faces))

for face in faces:
    x, y, w, h = face['box']
    cv2.rectangle(image, (x, y), (x+w, y+h), (0, 255, 0), 2)

cv2_imshow(image)
```

```
Faces detected: 10
```



```
# Face Recognition
```

```
pip install deepface mtcnn opencv-python
```

```
Requirement already satisfied: deepface in /usr/local/lib/python3.12/dist-packages (0.0.96)
Requirement already satisfied: mtcnn in /usr/local/lib/python3.12/dist-packages (1.0.0)
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.12.0.88)
Requirement already satisfied: requests>=2.27.1 in /usr/local/lib/python3.12/dist-packages (from deepface) (2.32.4)
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.12/dist-packages (from deepface) (2.0.2)
```

```python
import cv2
from deepface import DeepFace
from google.colab.patches import cv2_imshow

# Load images
img1 = cv2.imread("/content/side2.jpg")
img2 = cv2.imread("/content/side3.jpg")

if img1 is None or img2 is None:
    raise FileNotFoundError("One or both images not found!")

# Face verification
result = DeepFace.verify(
    img1_path="/content/side2.jpg",
    img2_path="/content/side3.jpg",
    model_name="ArcFace",
    detector_backend="mtcnn"
)

# Extract results
distance = result["distance"]
threshold = result["threshold"]
verified = result["verified"]

# Add text on images
text1 = "Person 1"
text2 = f"Person 2 | Match: {verified}"

cv2.putText(img1, text1, (10,30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)

cv2.putText(img2, text2, (10,30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0,255,0), 2)

# Resize img2 to match img1's height while maintaining aspect ratio
h1, w1 = img1.shape[:2]
h2, w2 = img2.shape[:2]

if h1 != h2:
    aspect_ratio = float(w2) / float(h2)
    new_w2 = int(h1 * aspect_ratio)
    img2 = cv2.resize(img2, (new_w2, h1), interpolation=cv2.INTER_AREA)

# Combine images side by side
combined = cv2.hconcat([img1, img2])

# Show image
cv2_imshow(combined)

# Print difference info
print("🔍 Face Recognition Result")
print("---------------------------")
print("Verified (Same Person):", verified)
print("Distance (Difference):", round(distance, 4))
print("Threshold:", threshold)
```

```
🔍 Face Recognition Result
---------------------------
Verified (Same Person): True
Distance (Difference): 0.5197
Threshold: 0.68
```

PRACTICAL 8

AIM: Tensor flow operations

Code:-

Basic Arithmetic

```
import tensorflow as tf

a=tf.constant([10,20,30], dtype=tf.float32)
b=tf.constant([2,4,6], dtype=tf.float32)

print('Addition', tf.add(a,b).numpy())
print('Subtraction', tf.subtract(a,b).numpy())
print('Multiplication', tf.multiply(a,b).numpy())
print('Division', tf.divide(a,b).numpy())
```

```
Addition [12. 24. 36.]
Subtraction [ 8. 16. 24.]
Multiplication [ 20.  80. 180.]
Division [5. 5. 5.]
```

Trigonometric

```
angles=tf.constant([0.8, 3.1416/2, 3.1416], dtype=tf.float32)

print('Sine', tf.sin(angles).numpy())
print('Cosine', tf.cos(angles).numpy())
print('Tangent', tf.tan(angles).numpy())
```

```
Sine [ 7.1735609e-01  1.0000000e+00 -7.2399803e-06]
Cosine [ 6.9670671e-01 -3.6199901e-06 -1.0000000e+00]
Tangent [ 1.0296385e+00 -2.7624384e+05  7.2399803e-06]
```

Exponential & Logarithmic

```
vals=tf.constant([1.0, 2.0, 3.0], dtype=tf.float32)

print('Exponential', tf.exp(vals).numpy())
print('Logarithm', tf.math.log(vals).numpy())
```

```
Exponential [ 2.7182817  7.389056  20.085537 ]
Logarithm [0.        0.6931472 1.0986123]
```

Power & Root

```
print('Square', tf.square(vals).numpy())
print('Square Root', tf.sqrt(vals).numpy())
```

```
Square [1. 4. 9.]
Square Root [1.        1.4142135 1.7320508]
```

Comparison

```
x=tf.constant([5,10,15], dtype=tf.float32)
y=tf.constant([10,10,10], dtype=tf.float32)

print('Equal', tf.equal(x,y).numpy())
print('Greater', tf.greater(x,y).numpy())
print('Less', tf.less(x,y).numpy())
```

```
Equal [False  True False]
Greater [False False  True]
Less [ True False False]
```

2D Convolution

```
import tensorflow as tf

# Input: batch of 1 image (4×4) with 1 channel
x = tf.constant([
    [
        [[1], [2], [3], [4]],
        [[5], [6], [7], [8]],
        [[9], [10], [11], [12]],
        [[13], [14], [15], [16]]
    ]], dtype=tf.float32)  # shape = (1,4,4,1)

# 3×3 filter, 1 input channel, 1 output channel
filters = tf.constant([
    [
        [[1]], [[0]], [[-1]]
    ],
    [
        [[1]], [[0]], [[-1]]
    ],
    [
        [[1]], [[0]], [[-1]]
    ]], dtype=tf.float32)  # shape = (3,3,1,1)

y = tf.nn.conv2d(x, filters, strides=[1, 1, 1, 1], padding="VALID")
print(y.numpy())
```

```
[[[[-6.]
   [-6.]]

  [[-6.]
   [-6.]]]]
```

3D convolution

```
import tensorflow as tf

# Input: batch=1, depth=3, height=3, width=3, channels=1
x = tf.ones((1, 3, 3, 3, 1), dtype=tf.float32)

# 2×2×2 filter
filters = tf.ones((2, 2, 2, 1, 1), dtype=tf.float32)

y = tf.nn.conv3d(x, filters, strides=[1, 1, 1, 1, 1], padding="VALID")
print(y.numpy())
```

```
[[[[[8.]
    [8.]]

   [[8.]
    [8.]]]


  [[[8.]
    [8.]]

   [[8.]
    [8.]]]]]
```

Average pooling

```
import tensorflow as tf

# Input: 1 image (4×4) with 1 channel
x = tf.constant([
    [
        [[1], [2], [3], [4]],
        [[5], [6], [7], [8]],
        [[9], [10], [11], [12]],
        [[13], [14], [15], [16]]
    ]], dtype=tf.float32)

y = tf.nn.avg_pool(x, ksize=[1, 2, 2, 1], strides=[1, 2, 2, 1], padding="VALID")
print(y.numpy())
```

```
[[[[ 3.5]
   [ 5.5]]

  [[11.5]
   [13.5]]]]
```

Max pooling

```
import tensorflow as tf

x = tf.constant([
    [
        [[1], [2], [5], [4]],
        [[9], [3], [7], [8]],
        [[2], [6], [3], [1]],
        [[0], [4], [5], [2]]
    ]
], dtype=tf.float32)

y = tf.nn.max_pool(x, ksize=[1, 2, 2, 1], strides=[1, 2, 2, 1], padding="VALID")
print(y.numpy())
```

```
[[[[9.]
   [8.]]

  [[6.]
   [5.]]]]
```

PRACTICAL 9

AIM: Simple Example on Reinforcement Learning

Code:-

[3] 4s

```
!pip install Gymnasium
```

```
Requirement already satisfied: Gymnasium in /usr/local/lib/python3.12/dist-packages (1.2.3)
Requirement already satisfied: numpy>=1.21.0 in /usr/local/lib/python3.12/dist-packages (from Gymnasium) (2.0.2)
Requirement already satisfied: cloudpickle>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from Gymnasium) (3.1.2)
Requirement already satisfied: typing-extensions>=4.3.0 in /usr/local/lib/python3.12/dist-packages (from Gymnasium) (4.15.0)
Requirement already satisfied: farama-notifications>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from Gymnasium) (0.0.4)
```

[4] 0s

```
import numpy as np
import gymnasium as gym
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and sched
  return datetime.utcnow().replace(tzinfo=utc)
```

[5] 0s

```
# Create environment
env = gym.make("FrozenLake-v1", is_slippery=False)

# Initialize Q-table
state_size = env.observation_space.n
action_size = env.action_space.n
Q = np.zeros((state_size, action_size))

# Hyperparameters
learning_rate = 0.8
gamma = 0.95          # Discount factor
epsilon = 1.0         # Exploration rate
max_epsilon = 1.0
min_epsilon = 0.01
decay_rate = 0.005

# Training parameters
episodes = 1000
max_steps = 100
```

[6]
0s

```python
# ----------------------------
# Training (Q-Learning)
# ----------------------------
for episode in range(episodes):
    state, _ = env.reset()

    for step in range(max_steps):
        # Exploration vs Exploitation
        if np.random.uniform(0, 1) < epsilon:
            action = env.action_space.sample()  # Explore
        else:
            action = np.argmax(Q[state, :])     # Exploit

        # Take action
        new_state, reward, done, _, _ = env.step(action)

        # Q-learning update formula
        Q[state, action] = Q[state, action] + learning_rate * (
            reward + gamma * np.max(Q[new_state, :]) - Q[state, action]
        )

        state = new_state

        if done:
            break

    # Reduce exploration rate
    epsilon = min_epsilon + (max_epsilon - min_epsilon) * np.exp(-decay_rate * episode)

print("Training completed!\n")
```

```
Training completed!
```

[7]
0s

```python
# ----------------------------
# Testing the trained agent
# ----------------------------
state, _ = env.reset()
env.render()

for step in range(20):
    action = np.argmax(Q[state, :])
    state, reward, done, _, _ = env.step(action)
    env.render()

    if done:
        if reward == 1:
            print("Goal reached!")
        else:
            print("Fell into a hole!")
        break
```

```
Goal reached!
/usr/local/lib/python3.12/dist-packages/gymnasium/
  gym.logger.warn(
```

PRACTICAL 10

AIM: To study Reinforcement Learning Concept on Cart-Pole with DQN

a) To study Reinforcement Learning Concept on Cart-Pole with Random State:-

Code:

```
import gymnasium as gym
import random

env = gym.make("CartPole-v1")

def Random_games():
    for episode in range(10):
        env.reset()
        for t in range(500):
            env.render()
            action = env.action_space.sample()
            next_state, reward, terminated, truncated, info = env.step(action)
            print(t, next_state, reward, terminated, truncated, info, action)
            if terminated or truncated:
                break

Random_games()
```

```
0 [-0.01338038  0.24375546 -0.04166248 -0.2703306 ] 1.0 False False {} 1
1 [-0.00850527  0.43944642 -0.0470691  -0.5758575 ] 1.0 False False {} 1
2 [ 2.8365463e-04  6.3519549e-01 -5.8586247e-02 -8.8298911e-01] 1.0 False False {} 1
3 [ 0.01298756  0.831062   -0.07624603 -1.1934994 ] 1.0 False False {} 1
4 [ 0.0296088   0.63700587 -0.10011602 -0.9256555 ] 1.0 False False {} 0
5 [ 0.04234892  0.44336814 -0.11862913 -0.6660383 ] 1.0 False False {} 0
6 [ 0.05121629  0.25007847 -0.13194989 -0.41293672] 1.0 False False {} 0
7 [ 0.05621786  0.44679984 -0.14020863 -0.7441345 ] 1.0 False False {} 1
```

b) Code:-

[1]
8s

```
!pip install gymnasium stable-baselines3
```

```
Requirement already satisfied: gymnasium in /usr/local/lib/python3.12/dist-packages (1.2.3)
Collecting stable-baselines3
  Downloading stable_baselines3-2.7.1-py3-none-any.whl.metadata (4.8 kB)
Requirement already satisfied: numpy>=1.21.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium) (2.0.2)
Requirement already satisfied: cloudpickle>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium) (3.1.2)
Requirement already satisfied: typing-extensions>=4.3.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium) (4.15.0)
Requirement already satisfied: farama-notifications>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from gymnasium) (0.0.4)
Requirement already satisfied: torch<3.0,>=2.3 in /usr/local/lib/python3.12/dist-packages (from stable-baselines3) (2.10.0+cpu)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from stable-baselines3) (2.2.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from stable-baselines3) (3.10.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.3->stable-baselines3) (3.25.2)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.3->stable-baselines3) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.3->stable-baselines3) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.3->stable-baselines3) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.3->stable-baselines3) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch<3.0,>=2.3->stable-baselines3) (2025.3.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->stable-baselines3) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->stable-baselines3) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->stable-baselines3) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->stable-baselines3) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->stable-baselines3) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib->stable-baselines3) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->stable-baselines3) (3.3.2)
```

[4]
4m

```python
import gymnasium as gym
from stable_baselines3 import PPO

# Create environment with human render mode
env = gym.make("CartPole-v1", render_mode="human")

# Train agent
model = PPO("MlpPolicy", env, verbose=1)
model.learn(total_timesteps=10000)

# Watch the trained agent balance the pole
obs, _ = env.reset()
for _ in range(1000):
    action, _states = model.predict(obs)
    obs, reward, done, truncated, info = env.step(action)
    if done or truncated:
        obs, _ = env.reset()

env.close()
```

```
|    iterations          | 3           |
|    time_elapsed        | 132         |
|    total_timesteps     | 6144        |
| train/                 |             |
|    approx_kl           | 0.009994445 |
```